An In-Context Surrogate Model for Computational Fluid Dynamics (Report 2)

Brodie Knight
Supervised by Max Zhu, James Rowbottom, Stefania Fresca, Pietro Liò

Abstract

This project presents an in-context surrogate model for computational fluid dynamics (CFD), using a two-stage architecture. A separate context encoder is used to encode context tokens for the first stage, which is a transformer based dynamics model. The second stage detail refiner uses the flow matching architecture, to model the distribution of fluid states to predict. A novel data generation technique, *context switching*, is used to generate the fluid trajectories for training. This training set is compared to a standard training set by training the same model over both datasets, and the results demonstrate a clear advantage in robust in-context generalisation for the model trained with context switching.

1. Introduction

The first project report demonstrated that a patch based image transformer architecture with adversarial loss could successfully perform in-context learning on a small dataset with minimal artefacting. However, this architecture struggled to scale to a larger range of dynamics. In addition, while it appeared that the context encoder was learning to predict based on the provided context frames, in reality the model was very likely relying on spurious patterns in static fluid states to infer the context information (the encoded context tokens contained essentially no information). This report attempts to address these issues, by presenting a new, more interpretable, two stage model, an improved context encoder, and a novel approach to generating the datasets to make it infeasible for the context encoder to rely on such spurious correlations.

This report attempts to answer two questions. The first is to determine the effectiveness of the two-stage transformer dynamics base model and flow matching detail model architecture for CFD. The second is to determine how well a separate context encoder can genuinely represent the fluid parameters, in such a way that allows the model to generalise to unseen dynamics, and how to construct a training dataset to optimise for this.

The key finding of this report is that this in-context surrogate model finds it much easier to infer context through spurious relationships, rather than deeply learning the genuine dynamics which are invariably present for given context. In order to resolve this, a different kind of training dataset is created which incorporates *context switching* – the process of periodically switching the fluid parameters of a training trajectory.

The base dynamics model is now only trained on mean squared error (MSE) loss (with the adversarial loss removed), which intentionally generates some smoothing since the model is encouraged to collapse to the conditional mean. In this sense, this dynamics model attempts to learn and isolate the ‘deterministic’ part of the dynamics.

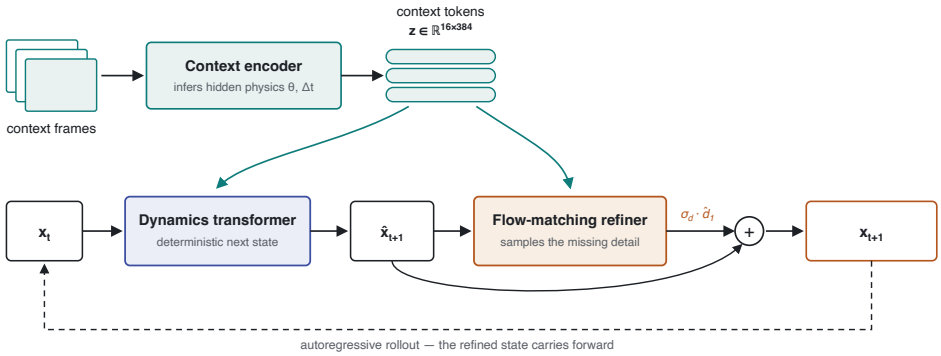


Figure 1: Overview of Model Architecture

Since the fluid behaves as a fundamentally stochastic process from the perspective of our coarse grid, we require a generative step, which generates the fine details according to a distribution, rather than fitting to the mean. Hence, the detail stage uses the flow matching [13] architecture to generate detail which learns to generate output which matches in distribution to the training dataset, thus solving the blurring issue that exists if using MSE loss.

The approach of splitting the problem into a deterministic base model and a refiner with uncertainty was initially shown to be successful in weather modelling [15, 16]. Recent work has used flow matching and diffusion models as machine learning surrogates [1, 3, 4, 10] for PDE modelling, which motivates the architectural choice of this report. Fan et al. [7] take a similar approach to this project for their two-stage fluid prediction pipeline, but use a hybrid neural solver rather than a fully learned dynamics solver for the base model. They also use a conditional diffusion model to capture the fine details, instead of a flow matching model (although they do not train for in-context learning).

Despite generally being applied to model the non-deterministic fluid features, this report demonstrates that flow matching is remarkably effective at *correcting* the destabilising artefacts that are introduced by the base dynamics transformer model.

2. Model Architecture

While the adversarial loss in the old model was effective in adding sharpness to the model output, when the model was applied to a larger training dataset, it struggled to scale effectively to a larger range of dynamics. Moreover, the old model placed a lot of the responsibility of modelling the fine details on the transformer model, which it is architecturally not well designed to handle.

The approach presented in this report attempts to resolve this by training two separate models. A high level overview of the overall architecture is presented in figure 1. The *dynamics stage* (stage 1) is the familiar patch image transformer architecture from the first report, without the discriminator network or adversarial loss. The *detail stage* (stage 2) is a flow matching model, which takes the output of the dynamics stage and adds detail according to a distribution learned from the training data. In theory, this approach aligns in essence with the framing of the Reynolds decomposition: the dynamic stage represents the deterministic, inter-cell information, and detail

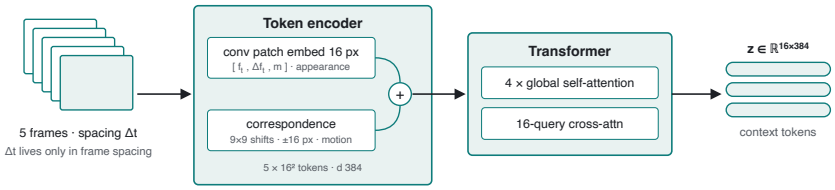


Figure 2: Context Encoder

stage represents the distribution of the chaos which cannot be modelled due to the coarseness of the data.

2.1. Context Encoder

Both dynamics and detail stages depend on the tokens produced by the context encoder, which encode the context that their output is conditioned on. The context encoder, as depicted in figure 2, accepts a set of $T = 5$ consecutive frames (separated by a constant timestep) and encodes 16×16 cell patches for each frame. Compared to the first report, this version of the context encoder embeds much richer information into the tokens which enter the context encoder transformer model, making the context easier to learn.

This new context encoder both encodes the frame, frame delta and mask information using a convolutional layer, as well as additively encoding *correspondence* of features across time. The correspondence layer seeks to quantify how detail moves between frames, and the implementation uses a very similar approach to FlowNetC (Dosovitskiy et al. [6]).

Each patch is encoded into the transformer as:

$$z_{t,ij} = W_{\text{ctx}}\ \text{vec}\Big(\big[\,\mathbf{f}_t \,\|\, \Delta\mathbf{f}_t \,\|\, m\,\big]_{\mathcal{P}_{ij}}\Big) + \Big(W_{\text{corr}} c_{t,ij} + b_{\text{corr}}\Big) + e^{\text{spat}}_{ij} + e^{\text{temp}}_{t}, \qquad \Delta\mathbf{f}_t = \begin{cases} \mathbf{f}_t - \mathbf{f}_{t-1}, & t > 0, \\ \mathbf{0}, & t = 0, \end{cases}$$

The first expression in the sum represents the convolutional layer, where $\mathcal{P}_{ij}$ denotes the (i, j)-th non-overlapping 16×16 patch, $\|$ is channel-wise concatenation, $\mathbf{f}_t \in \mathbb{R}^{4\times256\times256}$ is the t-th context frame ($t \in \{0, \ldots, T-1\}$), m is the binary geometry mask (set to 1 if the cell contains the fluid, and 0 if not), and $W_{\text{ctx}} \in \mathbb{R}^{d_{\text{ctx}}\times(16\cdot16\cdot9)}$ is a linear projection shared across all patches and frames. The inclusion of the frame delta, $\Delta\mathbf{f}_t$, trivially provides inter-frame dynamics that are useful for inferring context.

The second expression in the sum is the correlation layer encoding. This is calculated by first pooling the domain into a grid of $4 \times 64 \times 64$ cells, by downsampling into squares of 4×4, where each of the four channels in a square takes its respective square average. This is done to make the operations on the grid cheaper. Then, each cell is normalised by its euclidean norm, to give

$$\hat{g}_t(p) = \frac{g_t(p)}{\|g_t(p)\|}$$

where $g_t(p) \in \mathbb{R}^4$ is the cell average at cell p. After that, each normalised cell is correlated with nearby cells at the previous timestep, in order to determine if that information has been advected from that location in the previous frame. This is done via *cosine similarity*,

$$c_t(\delta, p) = \sum_{k=1}^{4} \hat{g}^{(k)}_t(p) \cdot \hat{g}^{(k)}_{t-1}(p + \delta)$$

which is a component wise sum over the products of matched previous and current cells, where $c_t(\delta, p)$ is the cosine similarity along direction δ between the previous and current frame, and $\hat{g}_t^{(k)}(p)$ is the value of channel k of the normalised pooled cell at position p. The offset directions, δ, are chosen to be within a 9×9 grid around the cell, yielding an $81 \times 64 \times 64$ tensor per context frame. This is further pooled into $c_{t,ij} \in \mathbb{R}^{81\times16\times16}$ to match the 16×16 token grid size, and encoded via a linear projection $W_{\text{corr}} c_{t,ij} + b_{\text{corr}}$, where b_{corr} is a shared bias. Since the correspondence term only spans a subspace of 81 dimensions, it should not crowd out the convolutional encoding term. Note, just as for the $\Delta\mathbf{f}_t$ term, the correspondence term is zero for the first frame since there is no preceding frame to compare it to.

The δ term has the capacity to reach up to 16 cells away from the source when calculating the cosine similarity. This means that if too large a Δt is used, or if the dynamics are such that details can move farther than 16 cells between frames, then the correspondence term ceases to provide meaningful information.

The $5\times16\times16$ patch tokens of size 384 each have learned spatial and temporal positional embeddings added (e_{ij}^{spat} and e_t^{temp} respectively), and pass through 4 transformer layers with global self-attention across the patch tokens. The 16 query tokens, also of size $d_{\text{ctx}} = 384$, are decoded from a standard perceiver layer [9], which is a cross attention layer between the query tokens and the patch tokens which have passed through the transformer model.

2.2. Dynamics Stage – Context Encoded Patch Transformer

The first stage of the generative model, the dynamics stage, is the backbone of the solver, responsible for solving the vast majority of the physics. The architecture of this model is illustrated in figure 3. This dynamics model (unlike the version presented in the first report) predicts the *delta change in fluid state* instead of the absolute fluid state. This change was made because as the training set grew, it became clear that the model no longer had the capacity to represent a sufficiently diverse distribution of different fluid states via its patch embeddings. Since the fluid state deltas lie on a smaller manifold than the absolute fluid states, it is much more manageable for the model to accurately output fluid deltas, because the latent space of the model able to allocate its capacity to a smaller range of potential outputs.

As discussed in the first report, this introduces the issue of autoregressive instability, because a predicted delta that is even slightly off can very quickly result in the fluid states degenerating into chaos. This was addressed by training over multiple timesteps in one iteration, and utilising back-propagation through time (BPTT) to teach the model how to 'self-correct' if its output is slightly misaligned. The detail refiner also helps to correct for this instability.

2.2.1. Encoder

The dynamics transformer model uses image patch embeddings, as in the context encoder, and embeds a single, fluid state that acts as the input from which to predict the next state. The patch embeddings for the dynamics are embedded via:

$$h_{ij} \;=\; W_{\text{dyn}}\,\text{vec}\Big(\big[\,\mathbf{x}_t \,\|\, m\,\big]_{\mathcal{P}_{ij}}\Big), \qquad \mathcal{T} \;=\; (h_{ij})_{i,j=1}^{16} \;\cup\; (g_k)_{k=1}^{16},$$

where $\mathcal{P}_{ij}$ denotes the (i, j)th non-overlapping 16×16 patch, $\mathbf{x}_t \in \mathbb{R}^{4\times256\times256}$ is the current fluid state (with the cells outside of the fluid mask set to zero), m is the binary geometry mask, and $W_{\text{dyn}} \in \mathbb{R}^{d\times(16\cdot16\cdot5)}$ is a linear projection shared across patches. The $g_k \in \mathbb{R}^d$ are 16 learned global capacity tokens appended to the sequence, giving $16^2 + 16$ tokens with $d_{\text{dyn}} = 384$. No additive positional embeddings are used, since the spatial position of the patch tokens are expressed via 2D rotary positional embedding (RoPE) [21].

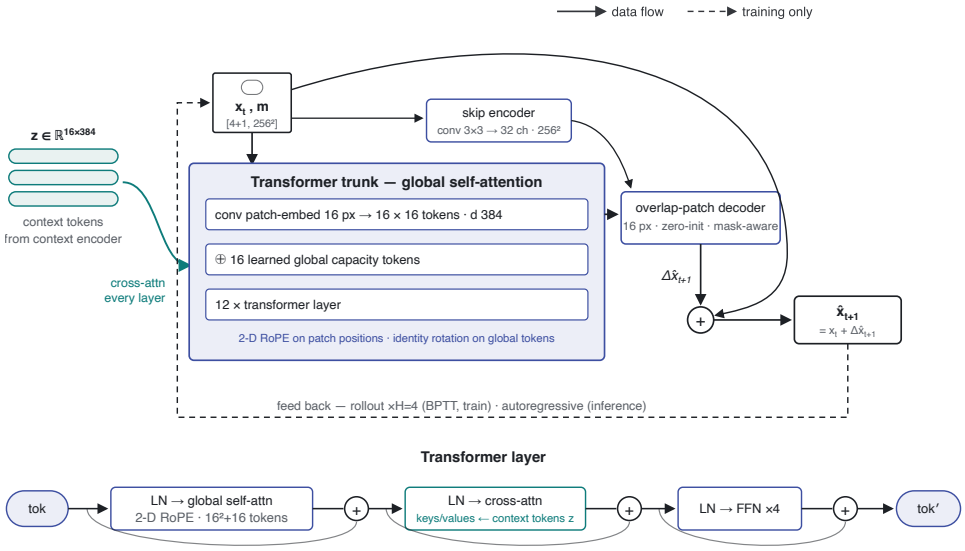


Figure 3: Patch Transformer Architecture

2.2.2. *Transformer Layer*

The transformer layer architecture used in this model is shown in figure 3. Each layer starts with a global self attention block, where the 2D RoPE embeddings are applied to the query and key matrices

$$\langle R_{ij}\, q,\ R_{i'j'}\, k\rangle \ =\ \langle R_{i-i',\,j-j'}\, q,\ k\rangle, \qquad R_{ij} \ =\ \mathrm{RoPE}_{2\mathrm{D}}(i,j), \quad R \equiv I \text{ for } g_k$$

so that attention logits depend on patch positions only through their relative offset $(i - i',\ j - j')$, ensuring translational invariance. The global tokens receive the identity rotation and so do not carry positional information.

The tokens then pass through a cross-attention sub-layer, allowing the $16^2 + 16$ generative tokens to attend to the 16 context tokens. This enables the information held within the context to inform the dynamics of the model. Finally, the generative tokens pass through a feed forward network (FFN) sub-layer, with an expansion ratio of $4 : 1$.

2.2.3. *Decoder*

The decoder block starts with a feed forward network, which decodes the token into a 16×16 grid of overlapping patches, each of size 24×24. Tent weightings are applied to values across the overlapping regions, with weights in the overlapping areas determined as a linear function from zero to one of distance from the overlap threshold. This can be expressed as

$$\hat{y}(q) \ =\ \frac{\sum_{i\in\mathcal{N}(q)} w_i(q)\,\hat{y}_i(q)}{\sum_{i\in\mathcal{N}(q)} w_i(q)}$$

where $\mathcal{N}(q)$ is the set of patches which contain point q, $w_i(q)$ is the per cell weight for patch i, and $\hat{y}_i(q)$ is the contribution from patch i.

A skip encoder is used to pull sharp details from the input directly into the output. This makes it much easier for the model to handle fine details, because the detail does not need to survive all 12 transformer layers, and the compression of the encoder/decoder. Despite predicting the delta instead of the absolute fluid state, the residual connection is still useful, because the fluid delta is geometrically similar to the absolute fluid state.

The cell values that are outside the fluid domain (inside collision geometry) may be set to arbitrary values by the model output. Hence, a masking step is applied to zero all cells outside the fluid domain before they enter the skip encoder. If this is not performed, it results in 'halo' artefacts around the collider boundaries because the convolutional layers would overlap with the cells that should have been masked but instead contain junk.

The skip encoder uses a thin 3×3 convolutional layer with 32 channels, with output $\mathbf{x}_{\text{skip},t}$. This is then fed, along with the 4 output channels from the patch decoded output, into a second, 64 channel 3×3 convolutional layer which decodes into an additive correction term. This correction term is then applied back to the patch decoded output, giving:

$$\Delta\hat{\mathbf{x}}_{t+1} = \hat{\mathbf{y}}_{t+1} + \texttt{post_conv}([\hat{\mathbf{y}}_{t+1}, \| \mathbf{x}_{\text{skip},t}])$$

which is added to the previous fluid state

$$\hat{\mathbf{x}}_{t+1} = \mathbf{x}_t + \Delta\hat{\mathbf{x}}_{t+1}$$

to give the final output of the dynamics model.

2.2.4. Loss

In order to stabilise over autoregressive rollouts, the model is trained with back-propagation through time (BPTT) with each training rollout consisting of $H = 4$ steps. Hence the loss function of the dynamics model is given as

$$\mathcal{L} = \frac{1}{H} \sum_{k=1}^{H} \frac{\ell(\hat{\mathbf{x}}_{t+k},\ \mathbf{x}_{t+k})}{\max(\ell(\mathbf{x}_t,\ \mathbf{x}_{t+k}),\ \epsilon)}, \qquad \hat{\mathbf{x}}_{t+k} = \Phi_\theta(\hat{\mathbf{x}}_{t+(k-1)},\ \mathbf{z})$$

with

$$\ell(\hat{y}, y) = \frac{\sum_q m(q) \left\| \hat{y}(q) - y(q) \right\|_2^2}{C \sum_q m(q)} + \lambda_1 \frac{\sum_q m(q) \left\| \hat{y}(q) - y(q) \right\|_1}{C \sum_q m(q)}$$

where q indexes the 256^2 spatial cell positions, $m(q) \in \{0, 1\}$ is the fluid mask ($m = 1$ on fluid cells, 0 inside solid geometry), $C = 4$ is the number of fluid channels and $\lambda_1 = 10^{-2}$ weights the L_1 term. Φ_θ denotes one autoregressive step of the model conditioned on the context tokens $\mathbf{z}$. The predictions are then fed back through the model, so that gradients propagate through all H steps. Each term is divided by the persistence baseline (the loss if the fluid is not changed at all), $\ell(\mathbf{x}_t, \mathbf{x}_{t+k})$. The term $\epsilon = 5 \times 10^{-3}$ guards against the case where the fluid is unmoving and the persistence baseline term goes to zero.

Without normalising the loss, the model would prioritise parameter updates for more difficult predictions where more has changed. Hence, if we had a sample in the training dataset where very little happens, there would be much less pressure to learn to adapt to this scenario, because the gradient updates would be dominated by samples where prediction is harder and the error is naturally higher. This means that the model would not learn to deal with 'easy' scenarios as effectively as it should.

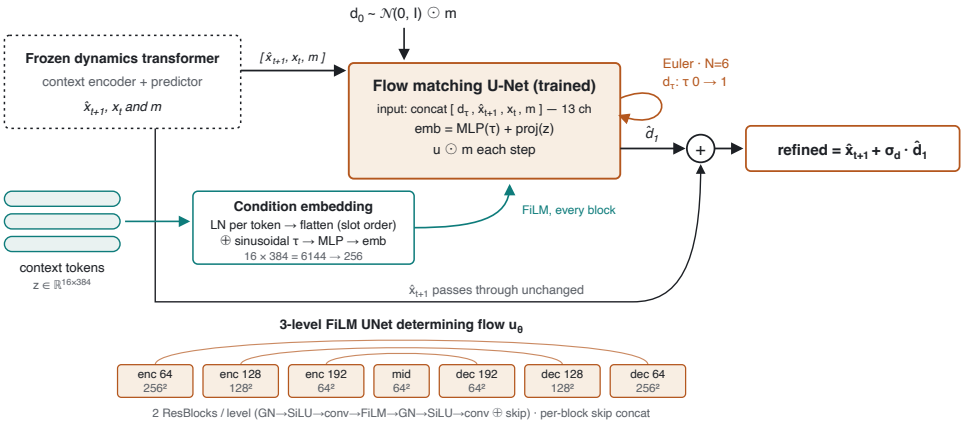


Figure 4: Conditional Flow Matching Architecture

2.3. *Detail Stage – Conditional Flow Matching Refiner*

The detail stage attempts to predict stochastic detail that the dynamics model cannot capture, due to its tendency to revert to the conditional mean. Flow matching models, introduced by Lipman et al. [13], are a type of continuous normalising flow (CNF) approach. They learn a velocity field that transports a multi-variate Gaussian, $\mathbf{d}_0 \sim \mathcal{N}(0, I)$ to a target distribution, $p(\mathbf{d}_1)$ by integrating an ODE described by the vector field $u_\theta(\tau, \mathbf{d} \mid c)$, over multiple steps. As such, they predict a distribution which is sampled by randomly sampling a point on the starting Gaussian $\mathbf{d}_0$, and transporting to a point in the destination domain along this ODE, such that $p(\mathbf{d}_1 \mid c)$ learns to approach the true conditional distribution.

Figure 4 shows the details of the flow matching implementation used in this project. For this model the training target of the detail model is the residual of the output of the dynamics stage against the ground truth fluid state, given by

$$\mathbf{d}_1 = \mathbf{x}_{t+1} - \hat{\mathbf{x}}_{t+1}$$

The distribution is conditioned on

$$c = (\mathbf{x}_t, \hat{\mathbf{x}}_{t+1}, \mathbf{z}, m)$$

where $\mathbf{x}_t$ is the (true or model generated) previous fluid state, $\hat{\mathbf{x}}_{t+1}$ is output of the dynamics model, $\mathbf{z}$ are the embedded context tokens and m is the geometry mask. In theory, because most of the difficult dynamics is taken care of by the dynamics model, the flow matching model is able to focus on what it is best at doing: learning the distribution of the residual.

At training time, $\mathbf{d}_0$ is drawn from the Gaussian distribution and matched with a training target, by first generating a frame using the base dynamics model and calculating the residual, $\mathbf{d}_1$. Points drawn from the starting Gaussian are at $\tau = 0$ and points at the target distribution sit at $\tau = 1$. The flow matching models draws a straight line segment from this $\mathbf{d}_0$ to $\mathbf{d}_1$, through $\tau \in [0, 1]$.

Rather than training to directly predict the output, the model learns the velocity field, $u_\theta(\tau, \mathbf{d} \mid c)$, via the training loss (masked by the fluid mask, m):

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{\tau, \mathbf{d}_0, c} \left\| u_\theta(\tau, \mathbf{d}_\tau \mid c) - (\mathbf{d}_1 - \mathbf{d}_0) \right\|_m^2, \qquad \mathbf{d}_\tau = \tau\, \mathbf{d}_1 + (1 - \tau)\, \mathbf{d}_0,$$

so that the velocity field at any point tends to the average of $(\mathbf{d}_1 - \mathbf{d}_0)$ over the training data.

Different methods of learning $u_\theta(\tau, \mathbf{d} \mid c)$ can be chosen depending on the nature of the distribution to be learned. In this case, a U-Net model [18] was chosen, as this is what is often used in image generation flow matching models. U-Nets are a neural network architecture which progressively embed in a hierarchical manner, from fine to coarse (encoder size), then from coarse to fine (decoder side), in an inverted 'U' shape. The key property of U-Nets is that they have skip embeddings which carry data across from the encoder side to the corresponding decoder level, meaning that high frequency details are not lost as data gets compressed through the bottleneck of the coarser layers.

A 13 dimensional concatenated frame, $[\mathbf{d}_\tau \,\|\, \mathbf{x}_t \,\|\, \hat{\mathbf{x}}_{t+1} \,\|\, m] \in \mathbb{R}^{13\times256\times256}$, is encoded into the U-Net, passed through, then decoded to a grid size of 4×256^2, which describes the vector field of the ODE, $u_\theta(\tau, \mathbf{d} \mid c)$. The U-Net possesses 3 layers of hierarchy, each with a FiLM sub-layer[17], which takes an embedded vector over all the context tokens and outputs both scale weightings and shift offsets for the convolutional layer. This layer architecture, presented by Dhariwal and Nichol [5], allows the context, $\mathbf{z}$ and flow time τ, to be able to amplify/augment the convolutional layers depending on the context it embeds. The FiLM layers and output head are zero-initialised, which means that the conditioning initially does nothing, and can slowly be learned as the model starts to rely on the context to reduce loss.

A scaling factor, $\sigma_d \in \mathbb{R}^4$, is also applied to the model input/output as a normalisation for each of the 4 channels. This is calculated as the standard deviation estimate of channel residual values, given what the base dynamics model predicts, after applying the geometry mask. This is done to make the job of the flow matching model easier, as it no longer needs to worry about scaling different channels from the starting distribution to the output distribution within the model itself.

At inference time, a random $\hat{\mathbf{d}}_0$ is drawn from the Gaussian distribution and passed through the model, by sampling the velocity field described by the U-Net (conditioned on c) six times and integrating over the ODE that these velocity samples describe via a simple Euler method. This gives the detail refiner residual output of $\mathbf{d}_1$, which is added onto the base dynamics model output to give

$$\mathbf{x}_{t+1} = \hat{\mathbf{x}}_{t+1} + \sigma_d \hat{\mathbf{d}}_1$$

where $\sigma_d \hat{\mathbf{d}}_1$ describes the channel wise scaling to give the final correction residual. This is the final model output for step $t + 1$ over the whole two-stage architecture.

3. Model Training

Two different training datasets were generated in order to evaluate the role of the context encoder. Dataset (A) was generated with standard fluid trajectories, and dataset (B) was generated using *context switching*, whereby the fluid parameters were changed throughout the trajectory to encourage the model to genuinely learn context.

A total of 2 972 160 fluid frames, over 24 768 fluid trajectories were generated for each dataset, each generated using the PyTorch based fluid solver developed by Zhu [24]. These data contain vastly more variety than those of the training dataset described in the first project report; the new data vary across collider geometry, fluid flow direction, boundary conditions and physical fluid parameters, and are generated in such a way to increase the pressure on the context encoder to learn context.

For each dataset, 344 different collider geometries were randomly initialised, by randomly positioning a set of $n \sim \mathcal{U}\{1, 2, 3, 4\}$ ellipses within the fluid domain. If multiple ellipses collide, or are too close to one another, then the positions are resampled until the constraints are satisfied.

3.1. Trajectory Samples

The context of a trajectory is made up of *fluid parameters*, the *timestep*, and *initial conditions*. The fluid parameters, θ_t, are drawn from the following distributions:

$$\begin{aligned}
\text{visc_model} &\sim \mathcal{U}\{\text{Newtonian}, \text{PowerLaw}, \text{Carreau}, \text{HerschelBulkley}\},\\
n_{\text{visc}} &\sim \mathcal{U}(0.3,\ 1.8), \qquad f_{\min} \sim \mathcal{U}(0.1,\ 0.6),\\
\gamma_{\text{scale}} &\sim \text{LogU}(2.0,\ 40.0), \qquad \mu \sim \text{LogU}(3 \times 10^{-4},\ 6 \times 10^{-2}),\\
\mu_{\text{bulk}} &\sim \text{LogU}(10^{-3},\ 3 \times 10^{-1}), \qquad \kappa \sim \text{LogU}(5 \times 10^{-7},\ 5 \times 10^{-3}),\\
C_v &\sim \mathcal{U}(1.5,\ 3.0), \qquad \gamma \sim \mathcal{U}(1.1,\ 1.67).
\end{aligned}$$

where visc_model selects the law that maps shear rate to effective viscosity, n_{visc} is the power law exponent, $f_{\min}$ is the floor on the viscosity multiplier as a fraction of μ (which prevents shear viscosity going to zero), γ_{scale} is the characteristic shear rate when the non-Newtonian dynamics activate, μ is the shear viscosity, μ_{bulk} is the bulk viscosity, κ is the thermal conductivity, C_v is the specific heat at constant volume and γ is the ratio of specific heats. The transport coefficients are drawn log-uniformly because they span two to four orders of magnitude, and a uniform draw would place almost all systems in the high-viscosity, high-conductivity regime. The viscosity model is categorical and three quarters of all systems are chosen to be non-Newtonian. The viscosity multiplier n_{visc} spans both shear-thinning ($n_{\text{visc}} < 1$) and shear-thickening ($n_{\text{visc}} > 1$) regimes.

The solver timestep, Δt, is also chosen randomly, so that the model can learn to evolve the fluid states over a continuum of timesteps. The context timestep is chosen to be distributed according to the log-uniform distribution:

$$\log(\Delta t) \sim \mathcal{U}(\log(0.01), \log(0.04))$$

since this range would generally obey the constraints required for the correspondence term in the context encoder to properly track features between frames. A log-uniform distribution is chosen because the problem difficulty scales multiplicatively with the timestep (i.e. moving from $\Delta t = 0.01 \rightarrow 0.02$ is equivalent in difficulty to moving from $\Delta t = 0.02 \rightarrow 0.04$), so a log-uniform distribution means that the ratio of relative timesteps should be uniformly distributed.

The set of *initial conditions* (ICs) describes the boundary conditions and initial fluid states for each trajectory, and are distributed according to:

$$\rho_\infty \sim \mathcal{U}(0.5,\ 2.0), \quad T_\infty \sim \mathcal{U}(80,\ 140), \quad v_\infty \sim \mathcal{U}(2,\ 7), \quad \alpha \sim \mathcal{U}(0^\circ,\ 360^\circ).$$

For each of the 344 geometries, 12 different starting contexts, and 6 different starting ICs are generated. A trajectory lasting for 120 frames is then generated for every combination of θ and IC draws. Hence, there are $12 \times 6 = 72$ trajectories per geometry and $344 \times 72 = 24\,768$ trajectories in total, resulting in a 2 972 160 total fluid frames.

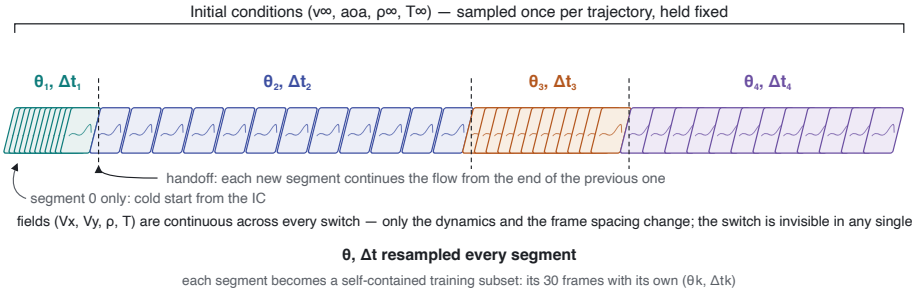


Figure 5: Single training trajectory with context switching.

3.2. Context Switching

Context switching, which is used in dataset (B), is the key technique used to encourage the context encoder to truly learn invariant features of the context instead of rely on spurious patterns. In an ordinary dataset (such as dataset (A)) the model can take shortcuts to identify the context, by observing patterns in static frames that correlate with context within the training data, but are not universal. Context switching within trajectories renders this shortcut no longer feasible for the model, as it would not result in the optimal path to minimise training loss.

Figure 5 illustrates how context switching works. Every trajectory that is generated has its dynamics parameters, θ, and the timestep, Δt, resampled after 30 frames of rollout by the numerical solver. Consequently, each trajectory can be split into segments, within which θ and Δt remain constant. The length of each segment was chosen to be 30 frames, to strike a balance between preventing spurious learning and allowing the model to observe realistic rollout trajectories. Note that the initial conditions (ICs) do not change along segments, but stay constant over an entire trajectory, otherwise this would result in the model being trained on dynamics that are very unusual.

Each segment from a single context switching trajectory acts as its own independent training run, which means that each step within a run is operated on by a *single context*. From the point of view of the model that is being trained, nothing special is needed, as it simply treats each segment/run as an independent trajectory with its own context.

3.3. Training Process

The training sweep for the dynamics model systematically progresses through each trajectory segment in the dataset. The process starts by sampling the frame at an offset of 5 as the first input frame (so that there is enough room for the initial context). A random context window is then chosen, which lies strictly before the frame we are trying to predict. These context frames are then given to the context encoder to encode. The model predicts the next $H = 4$ frames autoregressively, with the same shared context tokens. Once this is done, the gradients are back-propagated through time through all four iterations of the model. Gradients are also allowed to flow through the context encoder, which is trained simultaneously. Once one forward pass of 4 frames is completed, the model then moves onto frame 6, and the process continues, until the end of the trajectory segment is reached. Hence, most frames will be predicted four times per trajectory, as the prediction window slides through the trajectory segment.

The dynamics model and context encoder were jointly trained on 95% of their dataset, with a cosine-annealing learning rate of 1×10^{-4}. A held out validation dataset of the trajectories (which

included unseen mesh geometries) was used to observe model progress. The dynamics model was trained[1] for 60 epochs over the training dataset. Loss plots for the dynamics models are shown for both models (A) and (B) in figures 6, 7 respectively, with a comparison of validation losses between models in figure 8. Each figure shows the persistence normalised loss $\ell/\ell_{\text{persist}}$ (the same quantity that is used for the model training loss), since it offers a fairer metric on how the model performs, irrespective of prediction difficulty.

Superficial analysis of the loss comparison in figure 8 would suggest that model (B) performs worse. It trains both slower, and with persistently higher loss. However, this interpretation is misleading, because it fails to account for the differences in the respective datasets. The context switching dataset (B) is fundamentally harder to predict, due to the switching contexts. Consequently, there is a greater diversity of scenarios provided in dataset B, and the model requires more passes of the data to learn.

The detail model was trained on the output of the frozen dynamics model, over 40 training epochs. The detail model trains similarly to the dynamics model, except for the fact that it does not perform BPTT. The model steps forward a single frame per training iteration, chooses a preceding context window, and invokes the base dynamics model to generate $\hat{\mathbf{x}}_{t+1}$. A forward pass of the model is completed, and the residual is predicted, from which a loss is generated and the model trains. With a probability of $\frac{1}{4}$, the last frame output from the dynamics model is used as the next input frame, instead of the ground truth data, in order to train the model to handle compounding artifacts from the dynamics model.

The loss plots for models (A) and (B) are shown in figures 9 and 10 respectively. These models both exhibit very similar training dynamics, which suggest that the detail refiner model is learning to perform a similar role across both models.

4. Results and Analysis

The results presented (included at the end of the report) attempt to provide comprehensive insight into how the model performs. Hence, a variety of different fluid regimes are depicted, across many different collider geometries and across (v_x, v_y, ρ, T). Each strip presented depicts the autoregressive rollout of the labelled model ((A) or (B)) generated by iteratively feeding the model output back into itself, while keeping the context constant. The image strips also show the ground truth dataset to which the model output should be compared, as well as the context that was fed into the context encoder. Mean absolute error (MAE) plots are also included above, which plot $\text{MAE}(\hat{\mathbf{x}}_{t+1}, \mathbf{x}_{t+1})$ over rollout steps. The persistence baseline, $\text{MAE}(\mathbf{x}_t, \mathbf{x}_{t+1})$, is also included for reference. Importantly, none of the fluid trajectories presented were included in the training datasets of either model, so model success cannot be attributed to overfitting.

4.1. In-Context Learning

Comparison of model (A) (the model trained on dataset A) and model (B) (the model trained on dataset B) show differences in how they learn context, and how they generalise to unseen contexts. On the surface, the training vs. validation loss graph for model (A) (figure 6) and model (B) (figure 7) both appear to be healthy, despite model (B) taking substantially longer

[1]The model was trained on the Dawn cluster of Cambridge Centre for Scientific and Data Driven Discovery (CSD3) supercomputer. A single compute node with 8× Intel 1550 GPUs was used.

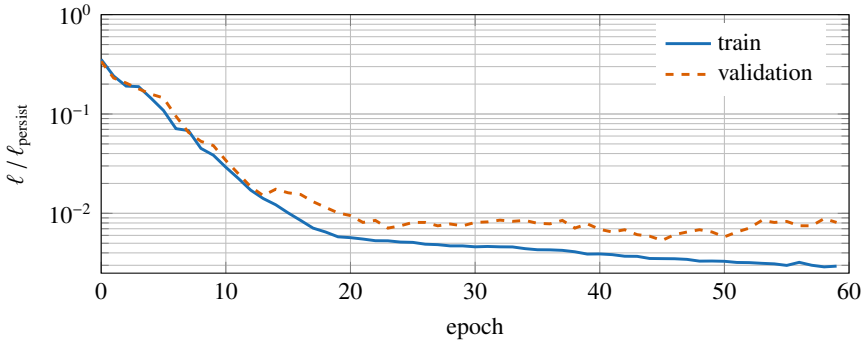


Figure 6: Model (A) (no context switching): persistence-normalised loss on training and held-out validation data.

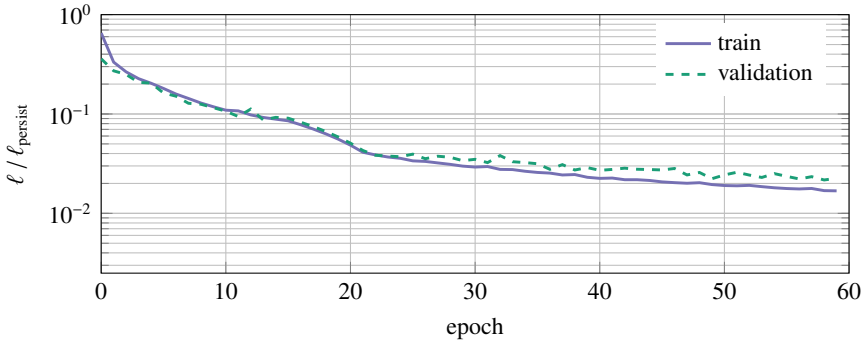


Figure 7: Model (B) (context switching): persistence-normalised loss on training and held-out validation data.

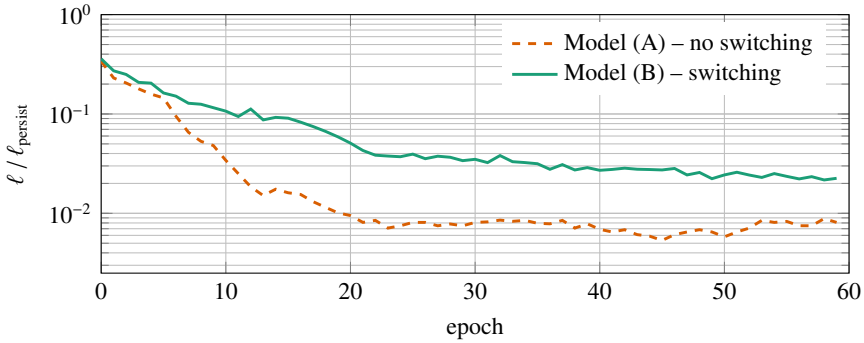


Figure 8: Held-out validation, persistence-normalised loss for both models.

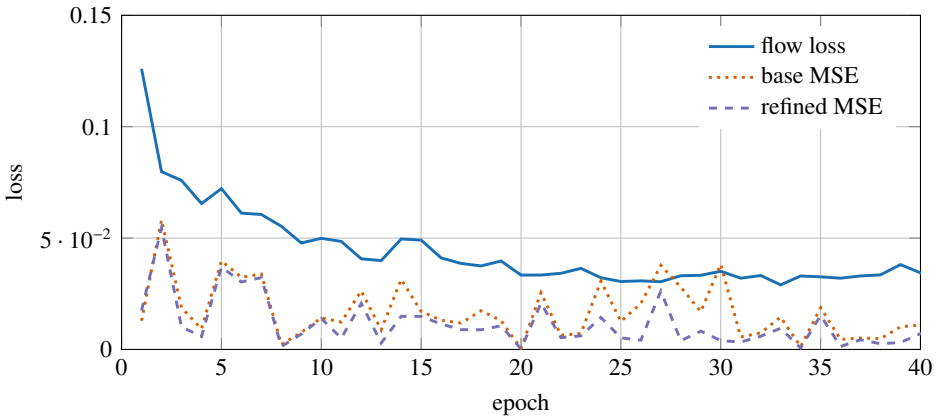


Figure 9: Flow matching loss for model (A) (without context switching)

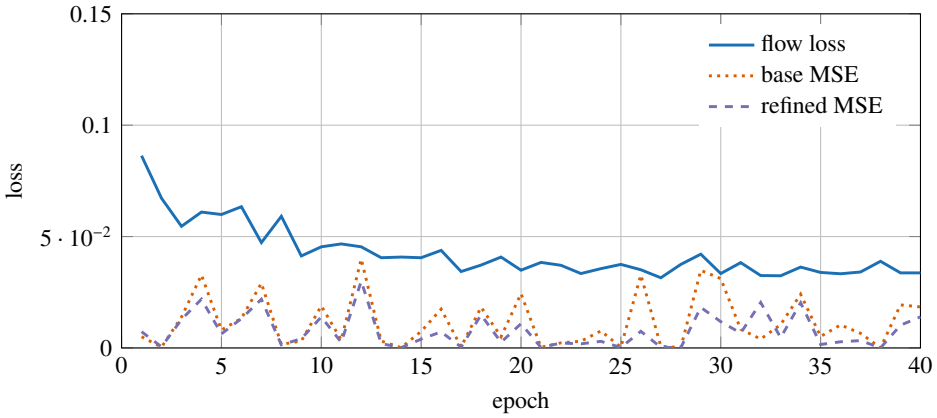


Figure 10: Flow matching loss for model (B) (with context switching)

to train than model (A). However, the held out validation set does not give the full picture of respective model performance.

Since each model is running on its *own* validation dataset, the trajectories that model (A) must deal with in its validation are distributed in the same way as the training data in dataset (A). Hence, model (A) is trained *and validated* on trajectories which all maintain the same parameters throughout, so it learns to *infer its context from static fluid features rather than the true dynamics observed by the context encoder*. This is exactly the problem that the context switching is designed to solve.

Figures 13 and 14 show the performance of both models (A) and (B) on held-out validation trajectories from dataset (A), which does not use context switching. Unsurprisingly, model (A) performs well on this task, because it was trained on similar trajectories. Model (B) also performs very well on this data, even though it was not directly trained on continuous trajectory rollouts, which is unsurprising because the distribution of trajectories in dataset (A) is, in a sense, a subset of those in dataset (B).

Similarly, figures 15 and 16 show output for models (A) and (B) when applied to a validation dataset in dataset (B) (where the trajectories use context switching). However, in this case there is a big difference in performance between the models – model (A) performs poorly, with its output being worse than the persistence baseline (the loss resulting from maintaining the old state). This is likely due to the spurious context inference problem. For both examples, the first context frame is the first frame in a trajectory segment, and so comes immediately after a context switch. The context frames clearly show a change in flow geometry, due to the new fluid parameters being applied. It would therefore make sense that if model (A) is relying on static patterns in order to infer the context, then it would be confused, because the static patterns belong to the context of the previous trajectory segment. This would explain why model (A) appears to 'mean-revert' which is exactly the behaviour to be expected if the model was unsure of the context, and attempts to 'hedge'. In contrast, model (B) handles both these scenarios excellently. It is clearly able to infer an accurate context encoding from the patterns that it observes across the context frames, which are *genuinely* a feature of the context, rather than relying on spuriously correlated patterns.

4.2. Detail Refiner Performance

The detail refiner flow matching stage was initially intended to allow the model to fit to distributional uncertainty, however it appears to also perform a significant *correcting* role. Figures 17 and 18 show the impact of the detail refiner for models (A) and (B) respectively. This specific trajectory was chosen because the discontinuity at the wave is difficult for the transformer model to represent, especially given that it is predicting the delta, so its output needs to superimpose perfectly onto the existing wave. When no refiner is active, the consequences of this are clear – visible ripples appear as artefacts of slightly misplaced features in the output of the dynamics model. However, the detail refiner is able to correct for this remarkably well. This is perhaps surprising, because the correction is a highly deterministic task, and is strictly conditional on the output of the model and the current fluid state.

In order to evaluate the ability of the detail refiner to actually carry out its intended role of capturing detail, a spectral energy ratio metric was recorded over the training process. A value of 1 means that the predicted frame has the same high frequency information metric as the ground truth frame. These metrics are presented in figures 11 and 12 for models (A) and (B) respectively. Interestingly, the detail refiner output seems to have a *lower* metric than the base dynamics model, which means that the refiner is *reducing* detail – completely opposite to what

it might be expected to do. The base dynamics model has an average spectral energy ratio of 1.0093 and 0.9946 for models (A) and (B) respectively, and the detail refiner model, these values are 0.9745 and 0.9842.

The most likely explanation for this is that the higher than expected values for the base dynamics models are the result of model artefacts. Therefore, the easiest way for the flow matching model to reduce its loss is to remove these artefacts, which results in a decrease in this metric.

This explanation is supported by the loss plots for both models (figures 9 and 10), which show the MSE for the base dynamics model prediction and the refiner model augmented prediction. It demonstrates that the refiner model is consistently able to reduce MSE by correcting the output of the dynamics model. This changes the framing of the two-stage model from a deterministic/non-deterministic split to a coarse dynamics/fine-detail split.

4.3. Performance on Out-of-Distribution Regimes

Perhaps the most useful property of an in-context model is its ability to generalise outside of its training distribution. In order to test this property, the models were tested on trajectories generated using a *non-monotonic* viscosity model, which was not included in the training set. This model output for these trajectories can be observed in figures 21 and 22.

The precise model used obeys a non-monotone flow with effective viscosity set by

$$\mu_{\text{eff}}(\dot{\gamma}) \;=\; \mu\, f(x), \qquad x \;=\; \dot{\gamma}/\dot{\gamma}_c,$$

and the flow curve given as

$$f(x) \;=\; f_{\min} \;+\; \frac{1 - f_{\min}}{1 + x^2} \;+\; (A - f_{\min})\,\frac{x^8}{x^8 + 1.5^8}, \qquad A \;=\; 5 + 45n$$

which 'jams' almost discontinuously at $1.5\dot{\gamma}_c$. Hence regions of fluid should be near solid above a certain critical rate. No existing viscosity model in the training set should be able to generate these dynamics, since all of the existing methods are monotone in their flow curve.

As expected, model (B) clearly outperforms model (A). However, it is perhaps surprising how well model (A) performs in this case. It seems to perform better when faced with a completely unseen regime, than when applied to the context switched trajectories (figures 15 and 16). In fact, this is entirely consistent with the hypothesis that model (A) learns to extract context from static fluid features. Since these trajectories ran without any context switching, model (A) could successfully derive context from static features which correlate with the fluid parameters.

4.4. Long Horizon Rollouts

Figures 19 and 20 demonstrate the models' performance on long horizon tasks. Both models are able to remain stable, without any noticeable artefacts appearing. Much of this can be attributed to the corrective nature of the detail model, which prevents the fluid state from decaying to a state that is too far off the training manifold. Furthermore, the 4-step BPTT at training time likely helps the dynamics model to handle its own output artefacts.

The mean energy plot in figure 19 suggests that for some long horizon rollouts there is relatively minor autoregressive drift. However for figure 20, mean energy density for models (A) and (B) have drifted by approximately -0.9% and $+0.6\%$ respectively by frame 80, and the drift does not seem to be slowing down. This is a classic problem with autoregressive surrogate modelling, as the models generally do not have a way to anchor macroscopic state information over a long rollout[2].

[2] In theory the context encoder may be used to do this, which is discussed briefly in the conclusion.

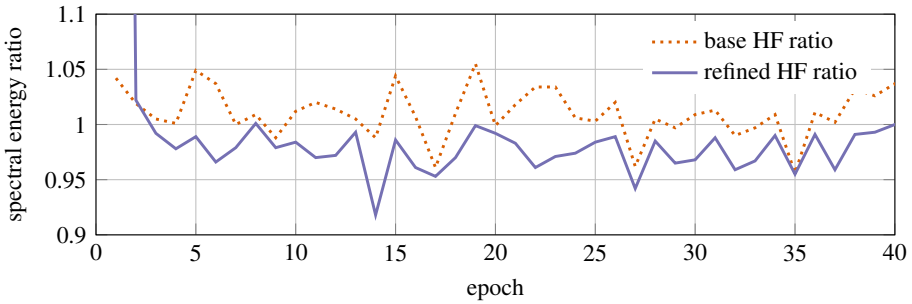


Figure 11: High frequency information over model (A) detail refiner training run

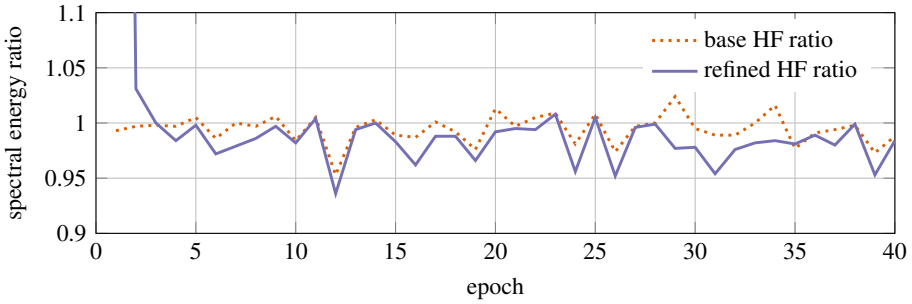


Figure 12: High frequency information over model (B) detail refiner training run

4.5. Wall-Clock Runtime Performance

One of the benefits of machine learning surrogate models is their ability to improve wall-clock simulation runtimes. Wall-clock time comparisons between the numerical solver used to generate the data and the model (both with and without the refiner) are presented in table 1. All figures presented were generated on an NVIDIA RTX 5090 GPU.[3] The numerical solver integrates at an average step of $\approx 1.07 \times 10^{-4}$ and so costs on average approximately 93 steps per 0.01s interval, whereas the model can progress this in a single step.

Since they are not limited by CFL constraints for stability (as explained in more detail in the first report), machine learning surrogates are able to operate with a much larger (in this case 93× larger) Δt between frames, although clearly this depends on both hardware and numerical solver efficiency. They are also able to better make use of the parallel architecture of the GPU, since the dense matrix multiplications used in machine learning scale very well on GPUs.

Notably, the detail refiner stage of the model occupies the majority of the wall-clock time. Further work could be done to investigate whether the number of iterations in the flow matching stage could be reduced, while preserving most of the model performance.

[3]Note: this is a different device to the Intel 1550 GPU nodes used for training, but was chosen due to the fact that it has CUDA and is more widespread.

Table 1: Wall-clock cost of advancing the flow state by $\Delta t = 0.01$, measured on an NVIDIA RTX 5090, using CUDA.

Method	Network evaluations	Wall clock	Speedup
Numerical solver (FVM)	—	5.74 s	1×
HFM	1	5 ms	1 148×
HFM + refiner	1 + 6	44 ms	130×

5. Comparison to Existing Models

The two key features of this model are the context switching, and the flow matching detail refiner. While several in-context CFD surrogate models exist, none use an approach analogous to context switching. Similarly, while there is existing literature applying flow matching and diffusion models to CFD, none apply a flow matching detail refiner on top of a dynamics transformer base. Kassaï Koupaï et al. [10] introduce ENMA, which utilises a flow matching architecture in an in-context model, and uses a transformer model to capture spatial dependencies across the fluid. However, their transformer model does not output a deterministic base fluid state to be corrected, but instead generates dynamics tokens, which are provided to condition the flow matching generative model, and they do not use a separate context encoder.

5.1. In-Context Models

Various existing in-context CFD surrogate models represent the context in slightly different ways, however none appear to use the separate context encoder design used by this project. For example, the ICON[23], VICON[2], Zebra[20] approach involves including sequential context frames in the input of the model (and ENMA includes the context in its conditioning transformer). This means that the model has full access to all context data for every prediction step that the model performs. Thus, the inferred context that these models capture is never compressed into a representation that allows clear quantification. This has benefits in theory, since it means that the model has the capacity to represent more complex and varied patterns in the context it uses to predict its output. However, this comes at the cost of interpretability when compared using a separate context encoder.

With a separate context encoder, it is possible to directly measure the information stored in the context tokens, which are by construction the only way that the dynamics model has access to the context information. This interpretability aspect was critical in setting the direction of this project. For the model trained on the old training dataset, which used standard training trajectories (without context switching), it was observed that the context contributed *almost nothing* to the output[4]. The context tokens that the context encoder was trained to output were approximately zero, and did not vary with different context inputs. This prompted the investigation into what process the model was using to manage to infer the context, without reliance on the context encoder. Without a separate context encoder, this insight would not have been available.

This invites the question of whether these other in-context surrogate models suffer from the spurious context inference problem that model (A) suffers from. As observed for model (A) (figure 6), the problem does not appear obvious if the validation data follow a trajectory which is

[4]Since the old model presented in the first report did not have a variable Δt, so it could infer all the information from just static frames. Model (A) does use its context tokens, presumably mainly to estimate Δt.

drawn from the same distribution as the training dataset. However, if the model were ever applied in cases where the fluid state may look like it came from a different trajectory with a different context to the actual context specified, it is likely that these models would perform poorly (as observed in this project). Neither ICON, VICON, Zebra nor ENMA address this issue explicitly nor implicitly through their chosen training regimes.

Another benefit from using a separate context encoder is efficiency. A separate context encoder means that the context only needs to be encoded once per rollout, rather than being recomputed per frame predicted. Over an entire autoregressive rollout, this removes a significant performance overhead that exists for these other models.

5.2. Refiner Models

CFD surrogate models differ substantially on how they approach the fact that there is uncertainty with how fluids behave. Many approaches, including Fourier neural operators (FNOs) such as Li et al. [12] and DeepONet (Lu et al. [14]) and foundation scale models such as Poseidon (Herde et al. [8]) and GPhyT (Wiesner et al. [22]) simply ignore the non-determinism, and treat it like a deterministic problem. This makes sense when we consider that we are training from a deterministic numerical solver. However, a benefit of machine learning is that it is probabilistic by nature, so machine learning models are the perfect candidate to learn a distribution over potential future fluid states.

Diffusion models, which iteratively de-noise towards a target, have been applied to CFD tasks [11, 19], although such models are very compute intensive both at training and inference time, as they require tens or hundreds of iterations to de-noise. Diffusion models seed their uncertainty on the initial starting Gaussian, which is then progressively decoded in such a way that the target distribution is sampled. The flow matching detail refiner used in this project performs an equivalent function, while requiring significantly less compute, since the network requires fewer evaluations to transport the noise to the target distribution. This is because flow matching models encourage the ODE to be substantially straighter, which requires fewer steps to integrate correctly, since the velocity along the paths is approximately constant.

For this project, the flow matching detail refiner does the majority of its work correcting for the fine details which the dynamics model struggles with. As such, the detail refiner performs a corrective role, which is similar in principle to FlowRefiner [4], which applies a flow matching model to 3D turbulence modelling to correct its own output. However, FlowRefiner operates as a single model performing both base prediction and correction steps, and it also does not operate as an in-context model.

6. Concluding Remarks

This report demonstrates that in-context learning can be an effective paradigm in CFD surrogate models. However, it is clear that great care must be taken in order to construct datasets which encourage the model to actually learn genuine physics rather than rely on spurious correlations. It would be an informative experiment to quantify the performance of other existing in-context CFD models on datasets generated using this context switching method. There is a reasonable chance that they would fail to infer context correctly for these datasets, because the data that they were trained on was similar in nature to dataset A.

Model (B) proved to be able to infer dynamics from a context window very effectively, regardless of the underlying static features of the fluid frames. However, because of the fact that

a separate context encoder is used, it is difficult to claim that this could operate as a deep 'in-context' foundation model in the sense that one could provide context for an arbitrary scientific computing method, and expect the model to solve it. Hence, it may be a better tool specifically for CFD modelling, but it does not fill the same role as models such as Poseidon [8] and GPhyT [22], which are true foundation models that have been trained over a much wider range of different physical systems.

An interesting direction of future work would be to investigate how it was possible that the flow matching detail model was able to reduce the MSE of the output so substantially. This is indicative that there is still capacity for additional deterministic prediction which is not being learned by the dynamics model. It could be that the model capacity to represent detail is saturated, and a greater token size (e.g. 512 dimensions) would allow the model to express the sufficient range of detail. Regardless, it is not necessarily a problem for inference performance, but it slightly undermines the framing of the role of each stage. However, if the detail model did not have to allocate its capacity to correcting the output of the dynamics model, it might be able to fulfil its original role and model the high frequency details more effectively.

Another potential area of future work would be to investigate to what extent the context can be used to prevent autoregressive drift. Drift of macroscopic features of the state (such as mean energy density) is observed to varying extents across different rollouts, which is problematic for CFD modelling. If the context could somehow be used to anchor these variables this could be very useful. This could potentially be trained by perturbing the input frames to change macroscopic quantities, and then training the model to correct.

While it is possible to effectively learn a subset of dynamics, training a fluid foundation model able to be universally applied to replace a numerical solver would require large quantities of data. As such, CFD surrogate models fill an important niche, due to their fast wall-clock speed, and their ability to perform in-context learning. However, it is unlikely that they will be able to fully replace what a numerical solver can provide in terms of interpretability, accuracy or reliability.

References

[1] Sahil Bhola and Karthik Duraisamy. Flow matching operators for residual-augmented probabilistic learning of partial differential equations. *arXiv preprint arXiv:2512.12749*, 2025. doi: 10.48550/arXiv.2512.12749.

[2] Yadi Cao, Yuxuan Liu, Liu Yang, Rose Yu, Hayden Schaeffer, and Stanley Osher. VICON: Vision in-context operator networks for multi-physics fluid dynamics prediction. *arXiv preprint arXiv:2411.16063*, 2024.

[3] Sipeng Chen, Junliang Liu, Hewei Tang, and Shibo Li. Multi-fidelity flow matching: Cascaded refinement of PDE solutions. *arXiv preprint arXiv:2605.16118*, 2026. doi: 10.48550/arXiv.2605.16118.

[4] Yilong Dai, Yiming Sun, Yiheng Chen, Shengyu Chen, Xiaowei Jia, and Runlong Yu. FlowRefiner: Flow matching-based iterative refinement for 3D turbulent flow simulation. *arXiv preprint arXiv:2604.17149*, 2026. doi: 10.48550/arXiv.2604.17149.

[5] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat GANs on image synthesis. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. doi: 10.48550/arXiv.2105.05233.

[6] Alexey Dosovitskiy, Philipp Fischer, Eddy Ilg, Philip Häusser, Caner Hazırbaş, Vladimir Golkov, Patrick van der Smagt, Daniel Cremers, and Thomas Brox. FlowNet: Learning optical flow with convolutional networks. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2758–2766, 2015. doi: 10.1109/ICCV.2015.316.

[7] Xiantao Fan, Deepak Akhare, and Jian-Xun Wang. Neural differentiable modeling with diffusion-based super-resolution for two-dimensional spatiotemporal turbulence. *Computer Methods in Applied Mechanics and Engineering*, 433:117478, 2025. doi: 10.1016/j.cma.2024.117478.

[8] Maximilian Herde, Bogdan Raonić, Tobias Rohner, Roger Käppeli, Roberto Molinaro, Emmanuel de Bézenac, and Siddhartha Mishra. Poseidon: Efficient foundation models for PDEs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. doi: 10.48550/arXiv.2405.19101.

[9] Andrew Jaegle, Felix Gimeno, Andy Brock, Oriol Vinyals, Andrew Zisserman, and João Carreira. Perceiver: General perception with iterative attention. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 4651–4664, 2021. doi: 10.48550/arXiv.2103.03206.

[10] Armand Kassaï Koupaï, Lise Le Boudec, Louis Serrano, and Patrick Gallinari. ENMA: Tokenwise autoregression for generative neural PDE operators. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. doi: 10.48550/arXiv.2506.06158. Spotlight.

[11] Georg Kohl, Li-Wei Chen, and Nils Thuerey. Benchmarking autoregressive conditional diffusion models for turbulent flow simulation. *Neural Networks*, 2026. doi: 10.1016/j.neunet.2026.108641.

[12] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021. doi: 10.48550/arXiv.2010.08895.

[13] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023. doi: 10.48550/arXiv.2210.02747.

[14] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3:218–229, 2021. doi: 10.1038/s42256-021-00302-5.

[15] Morteza Mardani, Noah Brenowitz, Yair Cohen, Jaideep Pathak, Chieh-Yu Chen, Cheng-Chin Liu, Arash Vahdat, Mohammad Amin Nabian, Tao Ge, Akshay Subramaniam, Karthik Kashinath, Jan Kautz, and Mike Pritchard. Residual corrective diffusion modeling for km-scale atmospheric downscaling. *Communications Earth & Environment*, 6(124), 2025. doi: 10.1038/s43247-025-02042-5.

[16] Jaideep Pathak, Yair Cohen, Piyush Garg, Peter Harrington, Noah Brenowitz, Dale Durran, Morteza Mardani, Arash Vahdat, Shaoming Xu, Karthik Kashinath, and Michael Pritchard. Kilometer-scale convection-allowing model emulation using generative diffusion modeling. *Science Advances*, 12:eadv0423, 2026. doi: 10.1126/sciadv.adv0423.

[17] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. doi: 10.1609/aaai.v32i1.11671.

[18] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, volume 9351 of *Lecture Notes in Computer Science*, pages 234–241. Springer, 2015. doi: 10.1007/978-3-319-24574-4$_2$8.

[19] Salva Rühling Cachay, Bo Zhao, Hailey James, and Rose Yu. DYffusion: A dynamics-informed diffusion model for spatiotemporal forecasting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. doi: 10.48550/arXiv.2306.01984.

[20] Louis Serrano, Armand Kassaï Koupaï, Thomas Wang, Pierre Erbacher, and Patrick Gallinari. Zebra: In-context generative pretraining for solving parametric PDEs. In *Proceedings of the 42nd International Conference on Machine Learning (ICML), PMLR 267*, 2025. doi: 10.48550/arXiv.2410.03437.

[21] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. doi: 10.1016/j.neucom.2023.127063.

[22] Florian Wiesner, Zoë J. Gray, Matthias Wessling, and Stephen Baek. Towards a physics foundation model. *arXiv preprint arXiv:2509.13805*, 2025. doi: 10.48550/arXiv.2509.13805. ICML AI4Physics Workshop, 2026.

[23] Liu Yang, Siting Liu, Tingwei Meng, and Stanley J. Osher. In-context operator learning with data prompts for differential equation problems. *Proceedings of the National Academy of Sciences*, 120(39):e2310142120, 2023. doi: 10.1073/pnas.2310142120.

[24] Max Zhu. Pytorch finite volume method solver. 2026. URL `https://github.com/Maxzhu123/fvm_solver`.

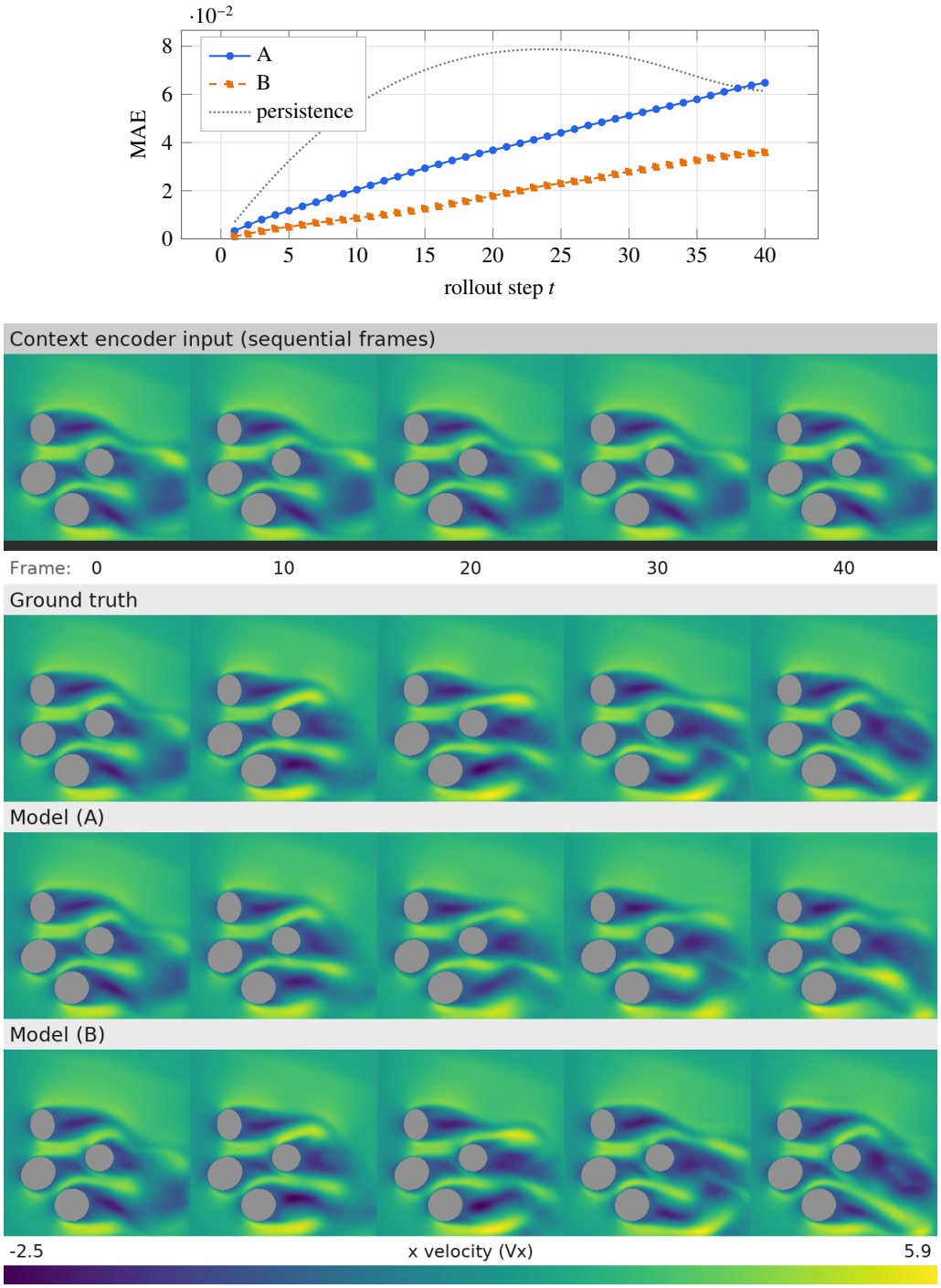


Figure 13: Model comparison of x velocity (v_x) on dataset (A) trajectory (no context switching), $\Delta t = 0.01$. Fluid parameters: Newtonian; $\mu = 0.000872$; $\mu_b = 0.00433$; $\kappa = 1.15 \times 10^{-5}$; $C_v = 1.58$; $\gamma = 1.29$; $\rho_\infty = 0.639$; $T_\infty = 113$; $|v_\infty| = 2.51$; $\alpha_\infty = 15.7°$.

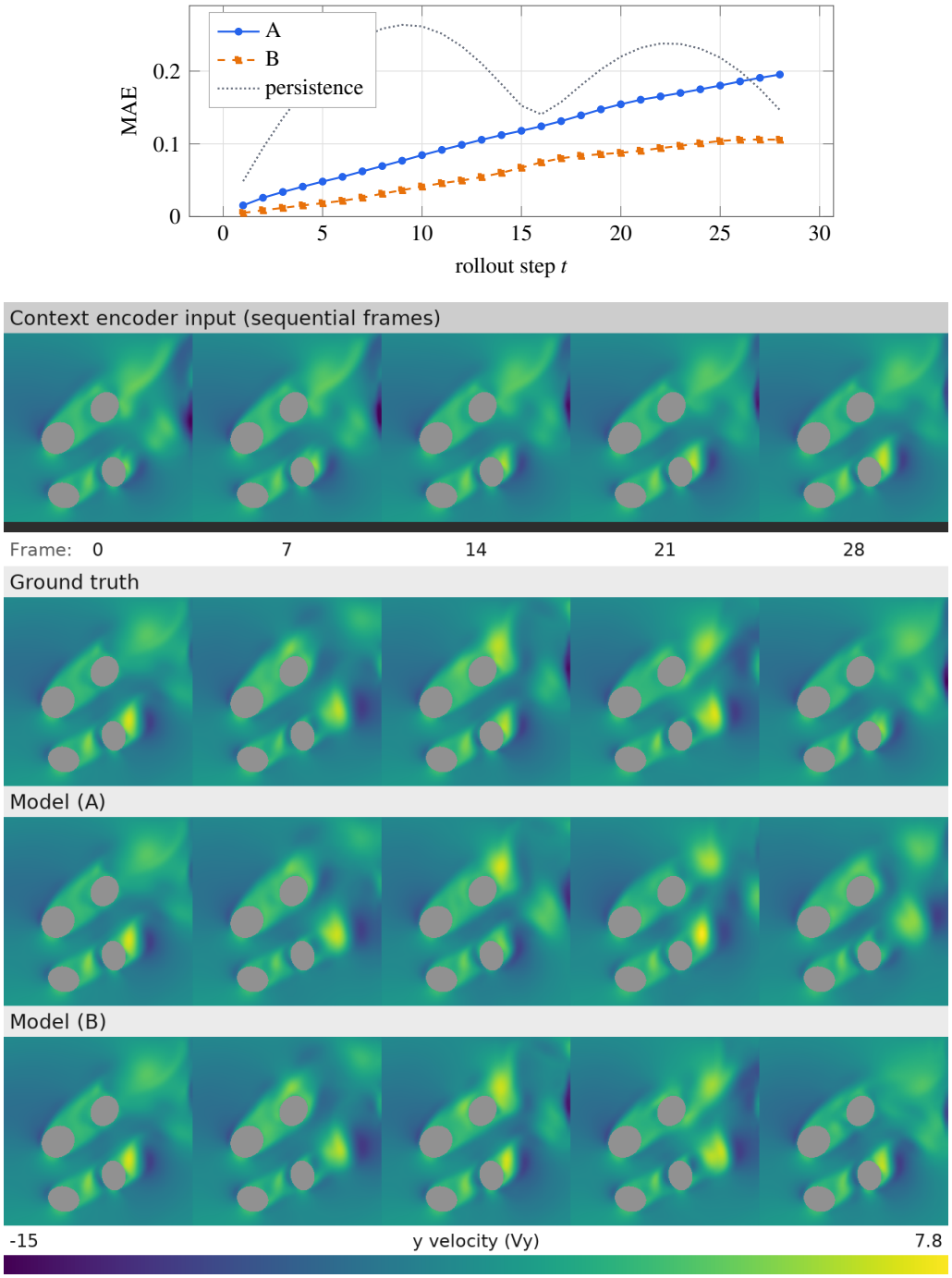


Figure 14: Model comparison of y velocity (v_y) on dataset (A) trajectory (no context switching), $\Delta t = 0.01$. Fluid parameters: PowerLaw ($n = 0.558$, $f_{\min} = 0.535$, $\dot{\gamma}_0 = 2.39$); $\mu = 0.0112$; $\mu_b = 0.046$; $\kappa = 0.000139$; $C_v = 1.59$; $\gamma = 1.66$; $\rho_\infty = 0.94$; $T_\infty = 124$; $|v_\infty| = 6.04$; $\alpha_\infty = 319°$.

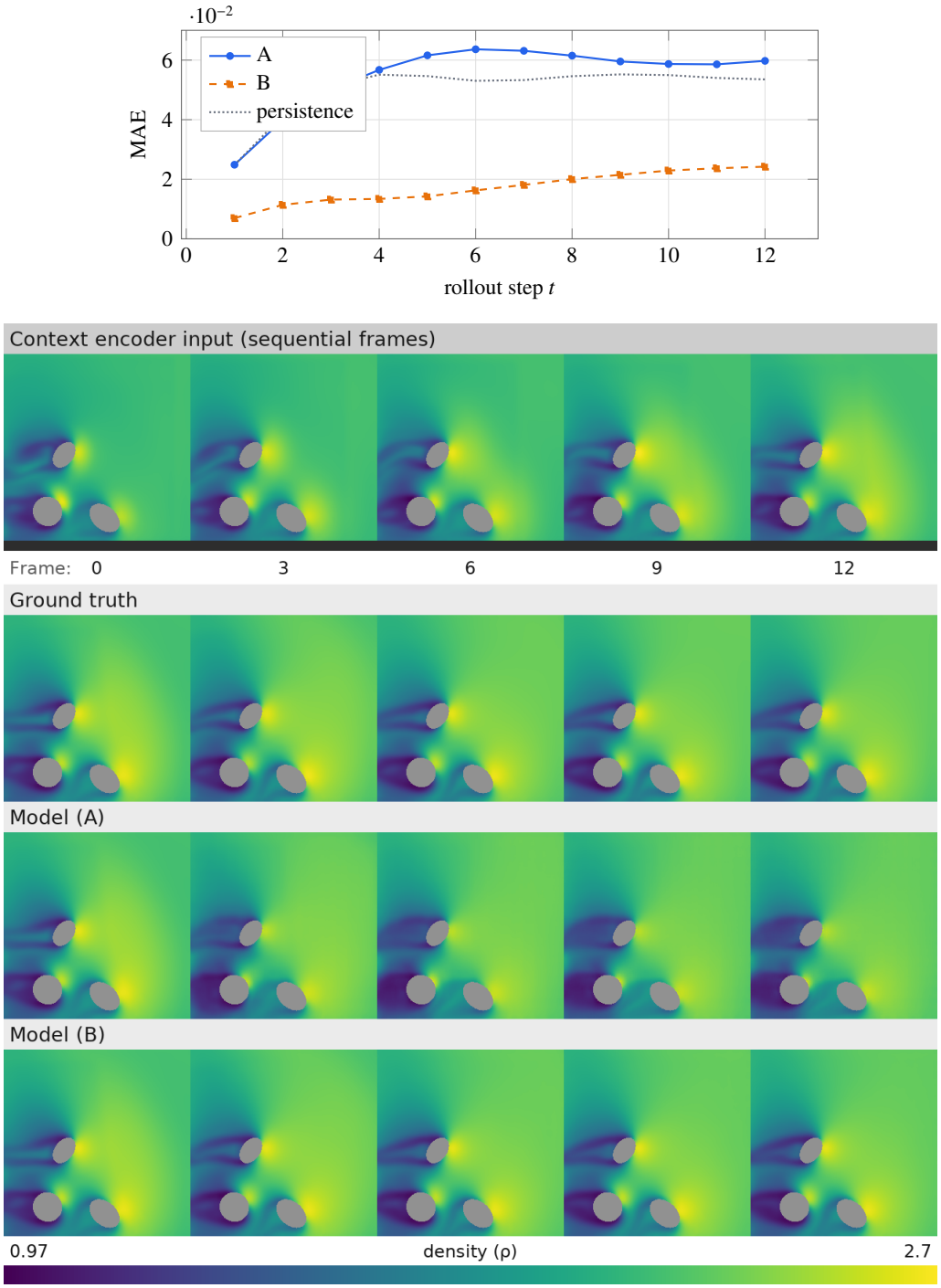


Figure 15: Model comparison of density (ρ) on dataset (B) trajectory (with context switching), $\Delta t = 0.0303$. Fluid parameters: HerschelBulkley ($n = 0.903$, $f_{\min} = 0.56$, $\dot{\gamma}_0 = 14.1$); $\mu = 0.0581$; $\mu_b = 0.259$; $\kappa = 7 \times 10^{-7}$; $C_v = 2.07$; $\gamma = 1.24$; $\rho_\infty = 1.99$; $T_\infty = 133$; $|v_\infty| = 6.37$; $\alpha_\infty = 167°$.

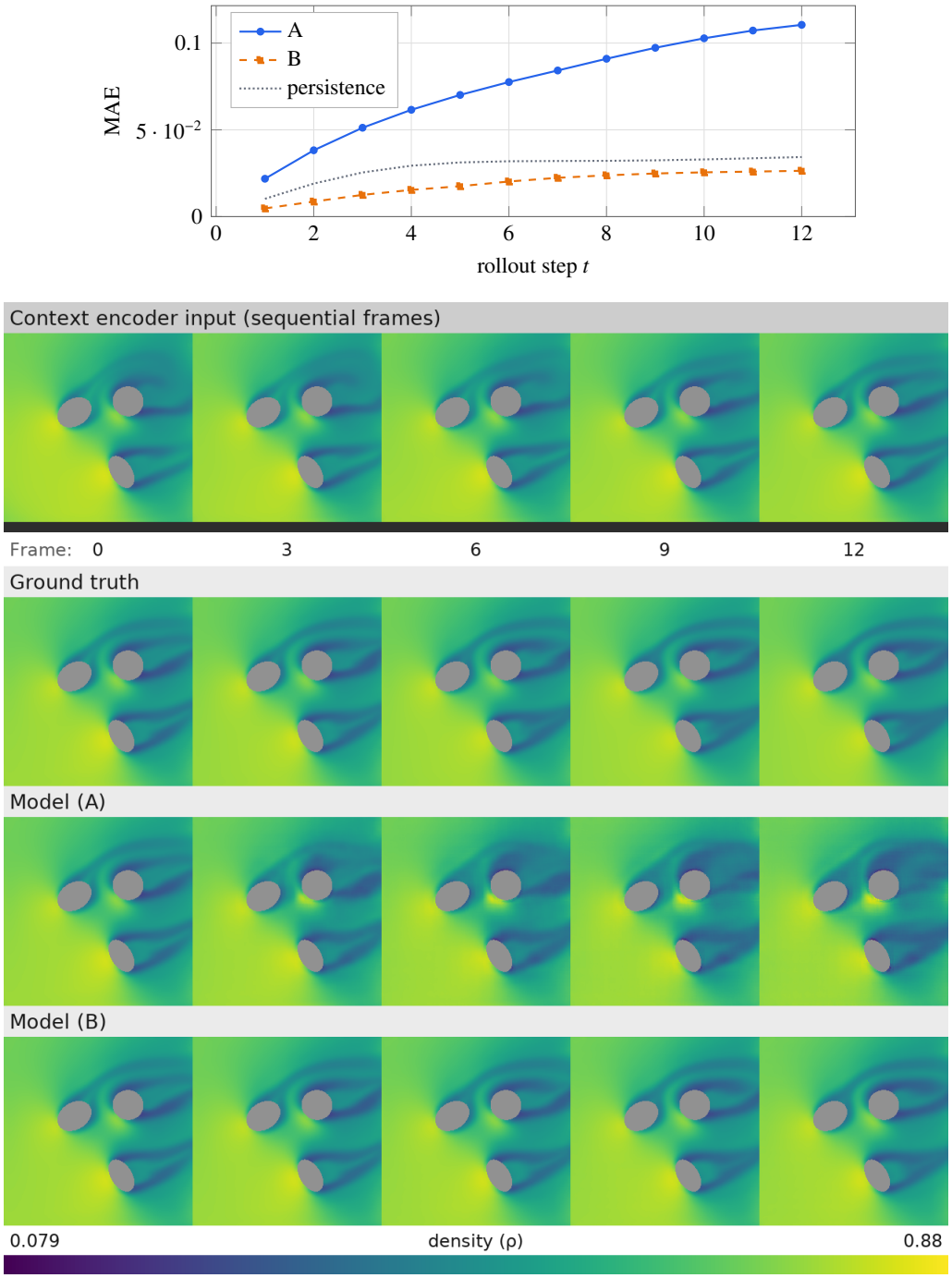


Figure 16: Model comparison of density (ρ) on dataset (B) trajectory (with context switching), $\Delta t = 0.0291$. Fluid parameters: Carreau ($n = 1.14$, $f_{\min} = 0.241$, $\dot{\gamma}_0 = 7.29$); $\mu = 0.0271$; $\mu_b = 0.235$; $\kappa = 6.13 \times 10^{-5}$; $C_v = 1.69$; $\gamma = 1.38$; $\rho_\infty = 0.634$; $T_\infty = 121$; $|v_\infty| = 6.83$; $\alpha_\infty = 337°$.

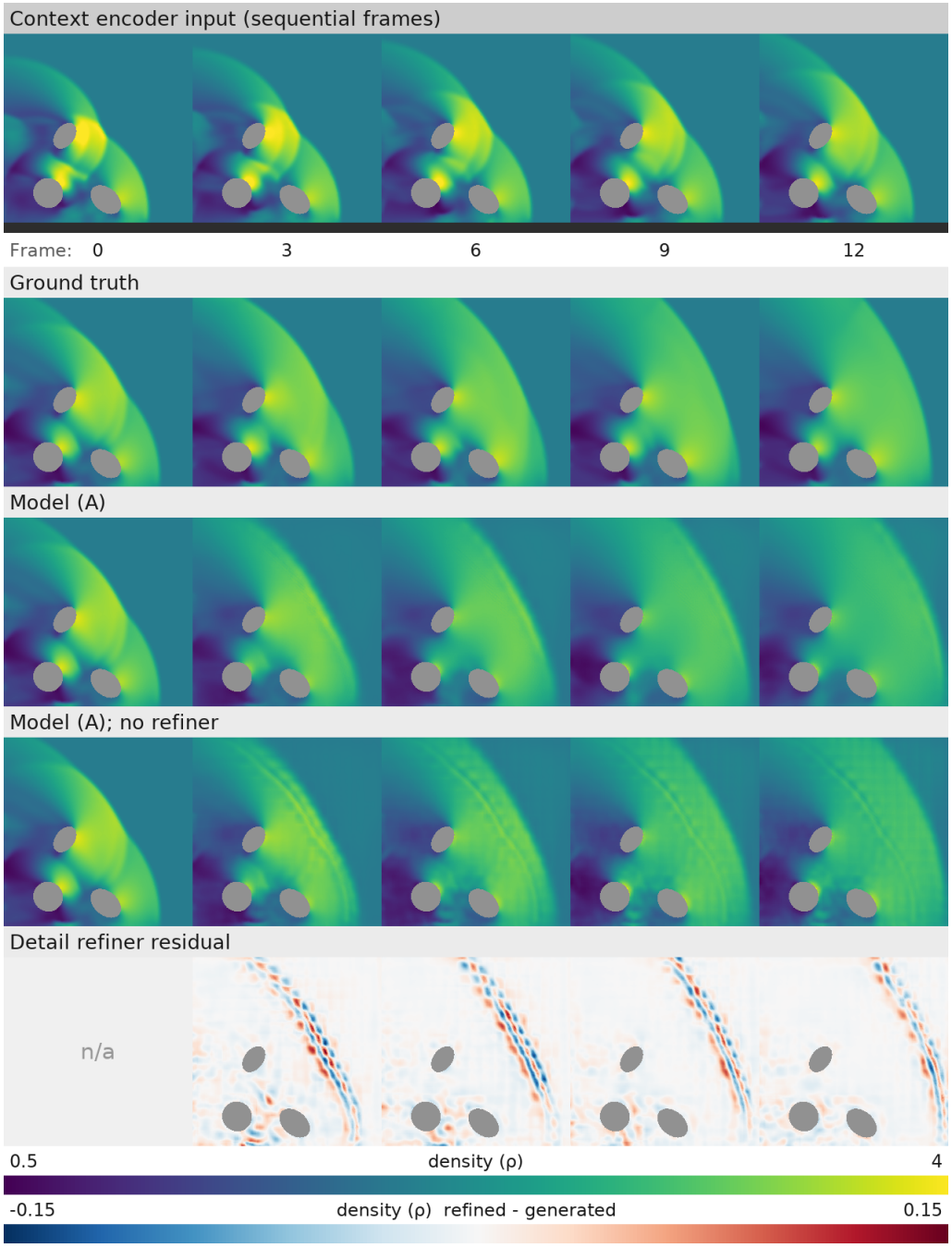


Figure 17: Autoregressive rollout of density (ρ) on dataset (B) trajectory (with context switching) under model `A` (with and without refiner active), $\Delta t = 0.019$. Fluid parameters: Newtonian; $\mu = 0.0067$; $\mu_b = 0.00265$; $\kappa = 1 \times 10^{-6}$; $C_v = 2.03$; $\gamma = 1.12$; $\rho_\infty = 1.99$; $T_\infty = 133$; $|v_\infty| = 6.37$; $\alpha_\infty = 167°$.

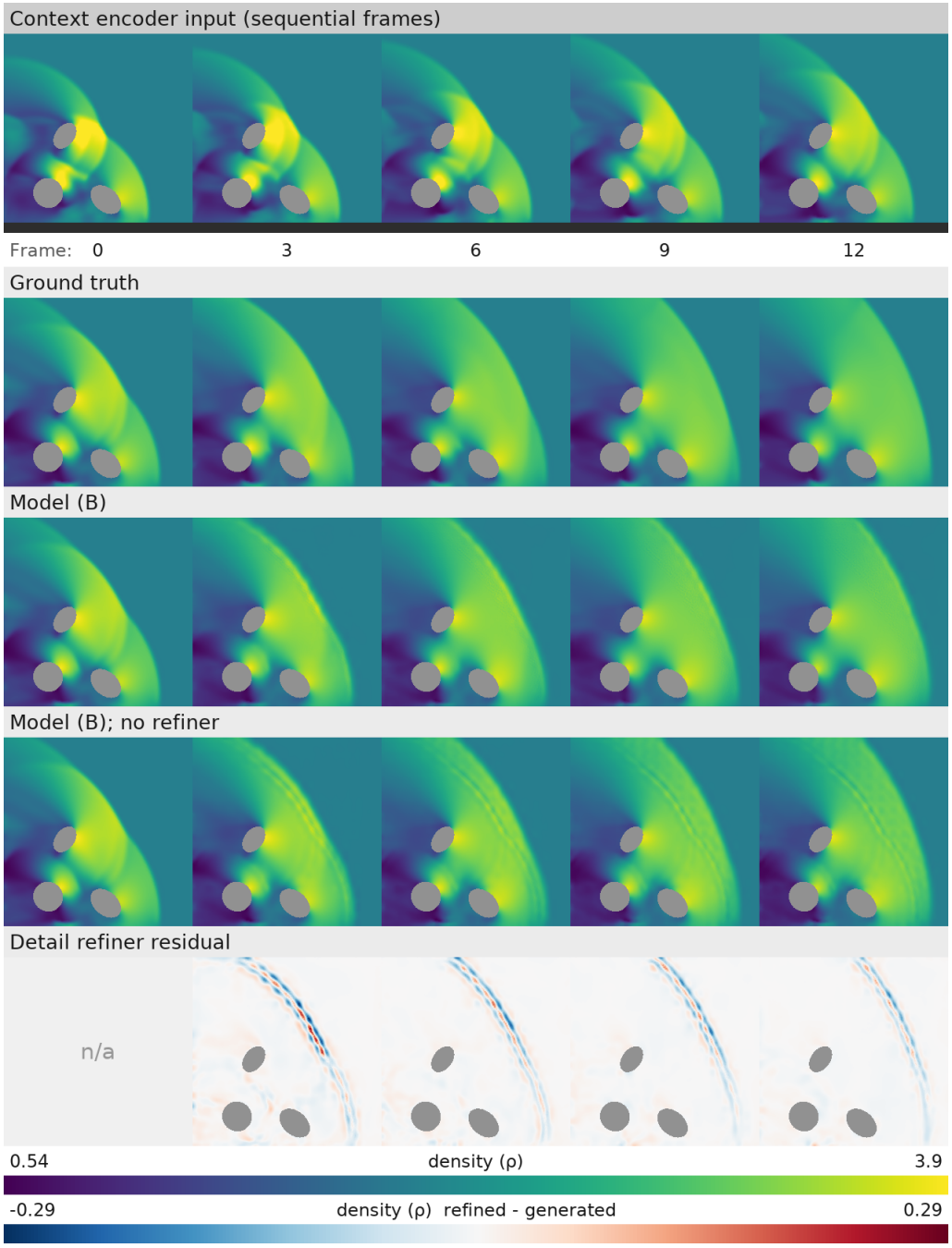
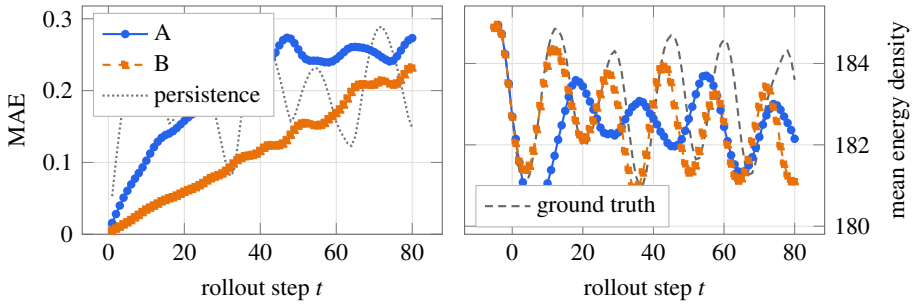


Figure 18: Autoregressive rollout of density (ρ) on dataset (B) trajectory (with context switching) under model B (with and without refiner active), $\Delta t = 0.019$. Fluid parameters: Newtonian; $\mu = 0.0067$; $\mu_b = 0.00265$; $\kappa = 1 \times 10^{-6}$; $C_v = 2.03$; $\gamma = 1.12$; $\rho_\infty = 1.99$; $T_\infty = 133$; $|v_\infty| = 6.37$; $\alpha_\infty = 167°$.



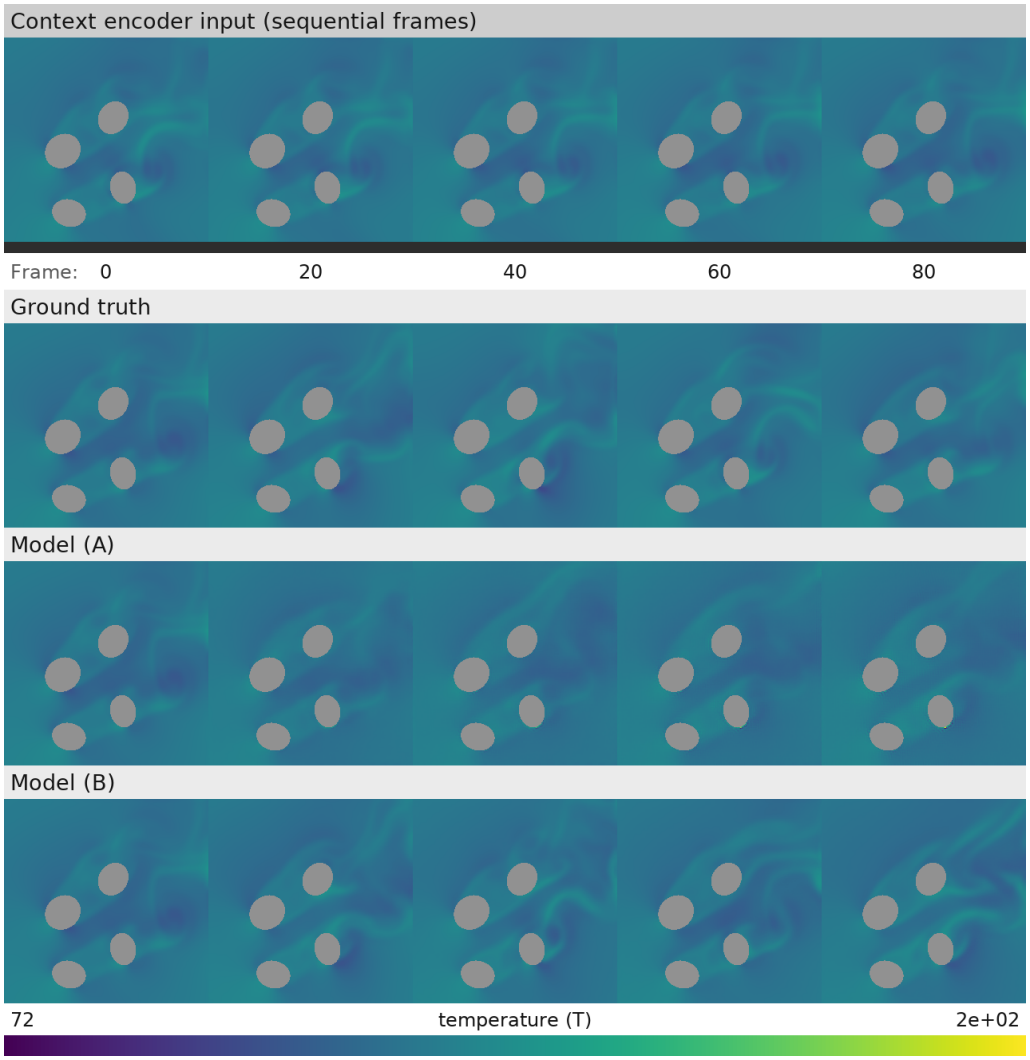


Figure 19: Model comparison of temperature (T) on long rollout, $\Delta t = 0.01$. Energy panel: mean density $\langle \rho C_v T + \frac{1}{2}\rho|v|^2 \rangle$. Fluid parameters: PowerLaw ($n = 0.558$, $f_{\min} = 0.535$, $\dot{\gamma}_0 = 2.39$); $\mu = 0.0112$; $\mu_b = 0.046$; $\kappa = 0.000139$; $C_v = 1.59$; $\gamma = 1.66$; $\rho_\infty = 0.94$; $T_\infty = 124$; $|v_\infty| = 6.04$; $\alpha_\infty = 319°$.

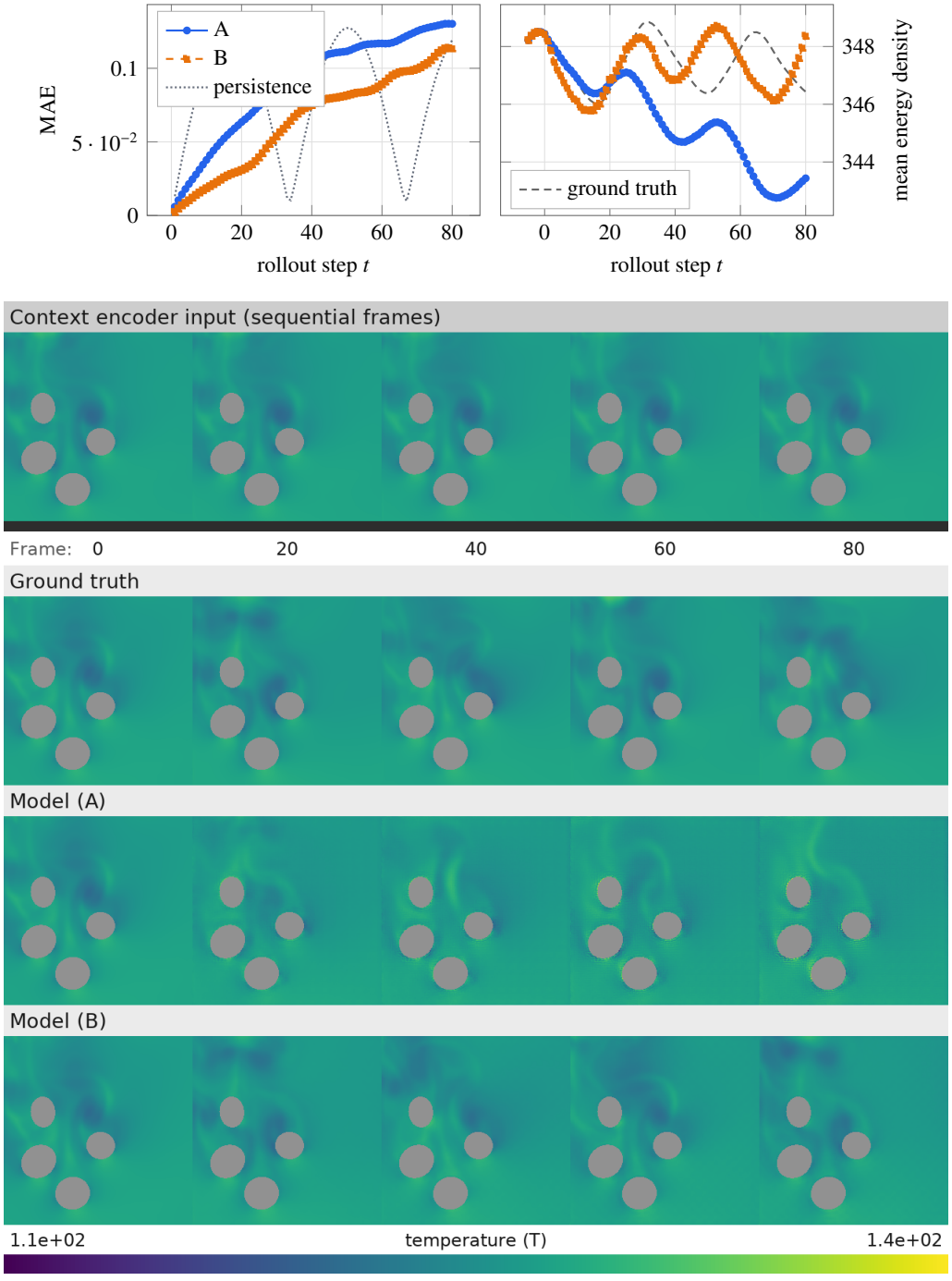


Figure 20: Model comparison of temperature (T) on long rollout, $\Delta t = 0.01$. Energy panel: mean density $\langle \rho C_v T + \frac{1}{2}\rho|v|^2 \rangle$. Fluid parameters: Newtonian; $\mu = 0.000872$; $\mu_b = 0.00433$; $\kappa = 1.15\times10^{-5}$; $C_v = 1.58$; $\gamma = 1.29$; $\rho_\infty = 1.93$; $T_\infty = 124$; $|v_\infty| = 2.88$; $\alpha_\infty = 249°$.

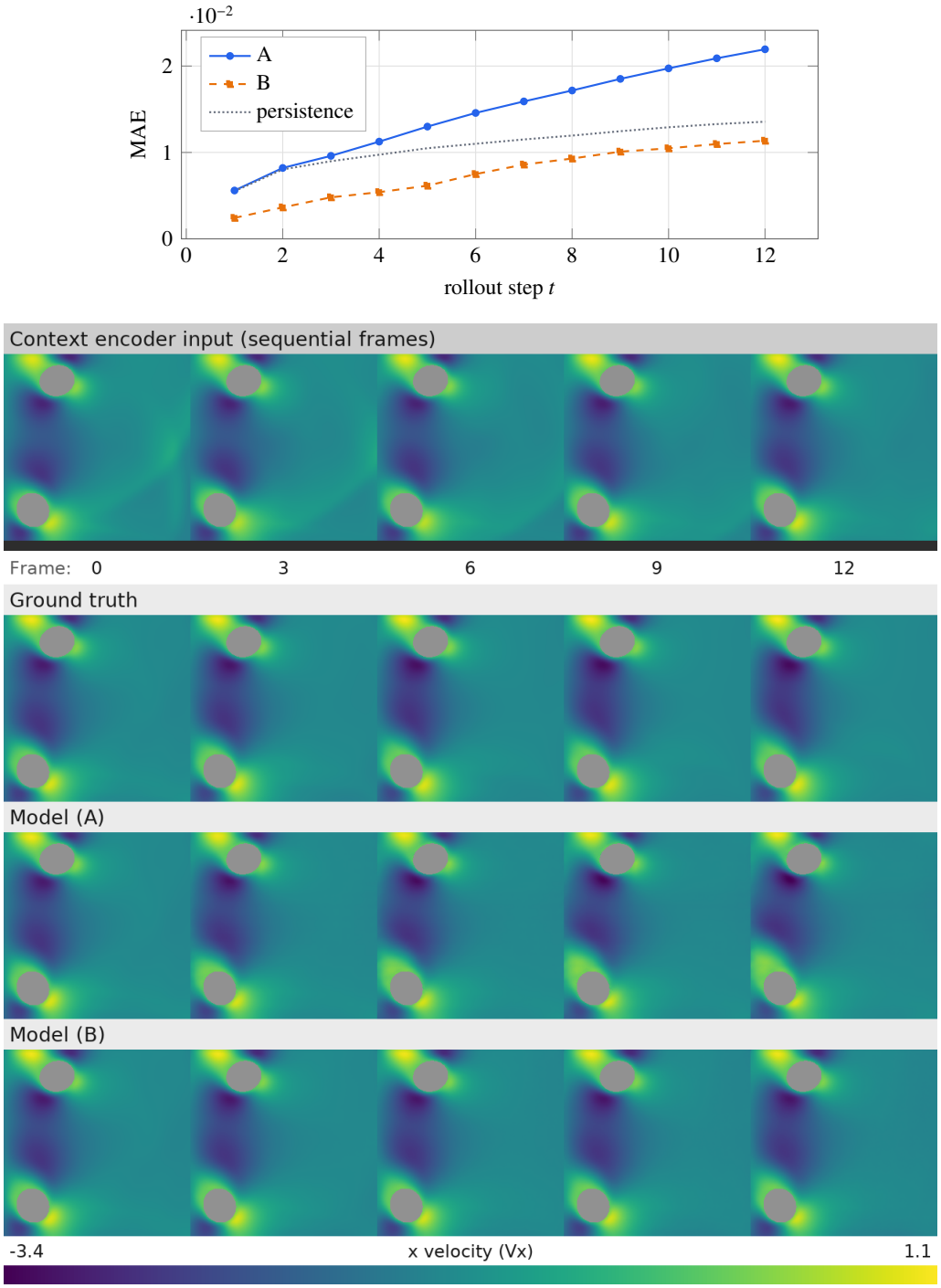


Figure 21: Model comparison of x velocity (v_x) on non monotonic regime, $\Delta t = 0.0193$. Fluid parameters: NonMonotonic ($n = 0.545$, $f_{\min} = 0.112$, $\dot{\gamma}_0 = 3.85$); $\mu = 0.00102$; $\mu_b = 0.0066$; $\kappa = 0.00018$; $C_v = 1.84$; $\gamma = 1.49$; $\rho_\infty = 0.664$; $T_\infty = 98.3$; $|v_\infty| = 2.59$; $\alpha_\infty = 236°$.

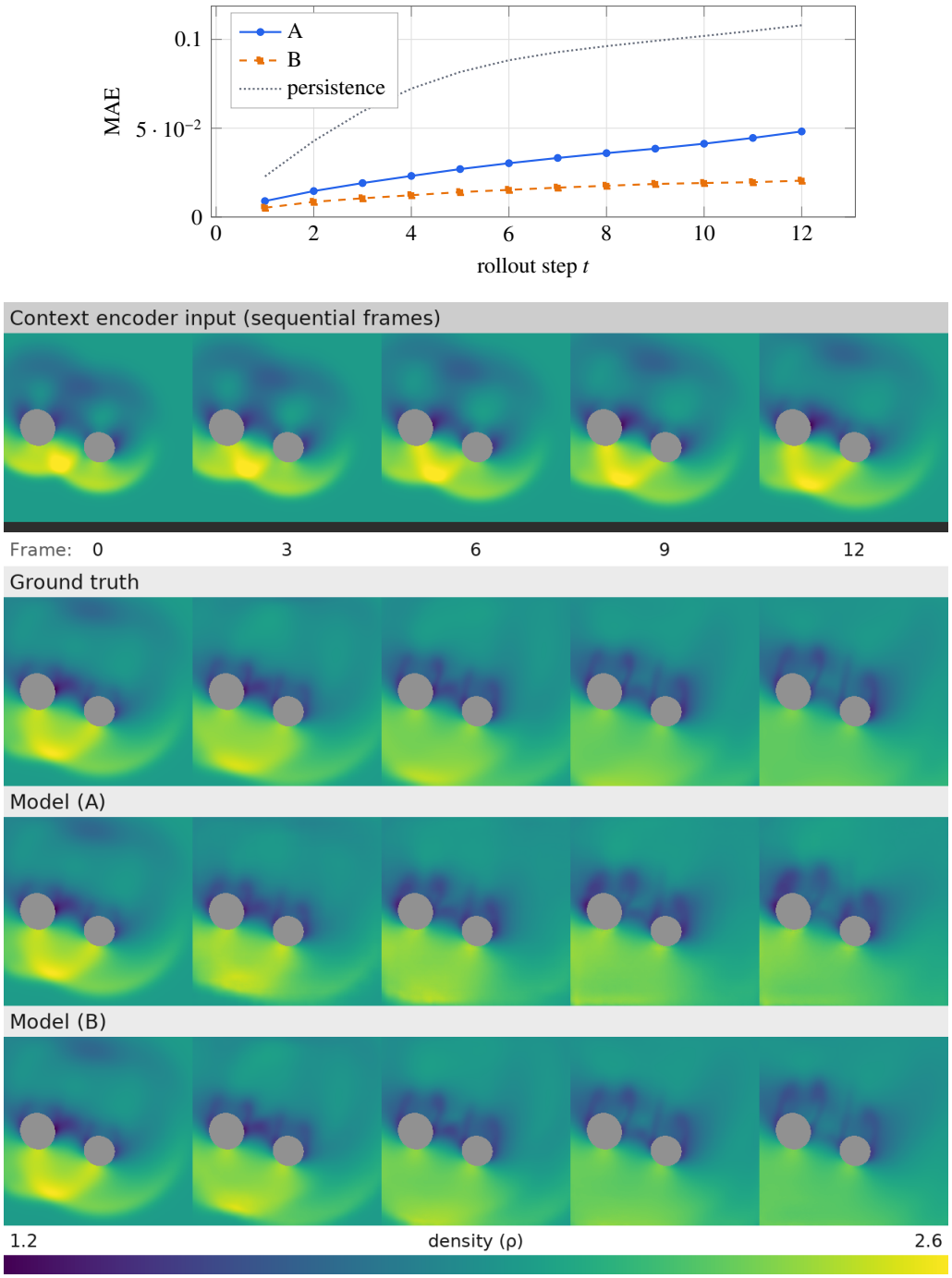


Figure 22: Model comparison of density (ρ) on non monotonic regime, $\Delta t = 0.0104$. Fluid parameters: NonMonotonic ($n = 0.482$, $f_{\min} = 0.544$, $\dot{\gamma}_0 = 4.84$); $\mu = 0.000813$; $\mu_b = 0.16$; $\kappa = 2.14 \times 10^{-5}$; $C_v = 2.19$; $\gamma = 1.17$; $\rho_\infty = 1.97$; $T_\infty = 84.4$; $|v_\infty| = 3.07$; $\alpha_\infty = 278°$.